CodeDumper

Turn any codebase into one organized Markdown document.

CodeDumper is a command-line tool that scans a project directory, builds a readable directory tree, collects eligible source and text files, and exports everything into a single, structured Markdown file.

Instead of manually opening dozens of files to share a codebase with an AI assistant, researcher, teammate, or reviewer, run one command and generate a project dump.

```
Project Directory
       |
       v
CodeDumper
       |
       +---- Directory Tree
       +---- File Discovery
       +---- Content Collection
       +---- Safety & Exclusion Checks
       |
       v
project-name.md
```

Features

- **One-command export:** Generate a complete Markdown snapshot of a project.
- **Readable directory tree:** Understand the structure before reading the source code.
- **Organized file sections:** Each file receives its own heading and code block.
- **Automatic project naming:** Generate `my-project.md` from the selected project directory.
- **Smart exclusions:** Skip common virtual environments, dependency directories, build artifacts, caches, and version-control internals.
- **Sensitive-file protection:** Exclude common secrets and credential files by default.
- **Binary-file detection:** Avoid dumping images, executables, and other unsuitable binary content.
- **Large-file protection:** Apply configurable size limits.
- **Custom exclusions:** Skip additional files or directories when needed.
- **Preview mode:** Inspect what would be processed without writing an output file.
- **Deterministic output:** Keep file ordering consistent for easier comparison and review.
- **Cross-platform CLI:** Designed for Windows, Linux, and macOS.
- **Offline operation:** Process local files without requiring an external API or cloud service.

Installation

Requirements

- Python 3.10 or later, subject to the version supported by the release.

- A terminal such as PowerShell, Command Prompt, Bash, or Zsh.

Option 1 — Install from the source repository

Clone the repository and install CodeDumper in editable mode:

```
git clone https://github.com/YOUR_USERNAME/CodeDumper.git
cd CodeDumper
python -m pip install -e .
```

Replace `YOUR_USERNAME` with the actual repository owner.

Verify the installation:

```
codedumper --help
```

Option 2 — Install a published release

Once CodeDumper is published to PyPI under the package name `codedumper`, users should be able to install it with:

```
python -m pip install codedumper
```

Then launch it from any terminal:

```
codedumper
```

The PyPI installation command is a planned distribution method; it works only after the package has been published under that name.

The CLI experience

CodeDumper should provide a clean, professional interface whenever a user types `codedumper`.

Launch the application

```
codedumper
```

Expected interface:

```
  ____          _      ____
 / ___|___   __| | ___|  _ \ _   _ _ __ ___  _ __   ___ _ __
| |   / _ \ / _` |/ _ \ | | | | | '_ ` _ \| '_ \ / _ \ '__|
| |__| (_) | (_| |  __/ |_| | |_| | | | | | |_) |  __/ |
 \____\___/ \__,_|\___|____/ \__,_|_| |_| |_| .__/ \___|_|
                                             |_|

                CODEDUMPER CLI
          Turn your codebase into Markdown.

  1. Dump the current project
  2. Dump another project
  3. Preview project contents
  4. Configure exclusions
  5. Help and documentation
  6. Exit

  Select an option [1-6]:
```

This is the proposed interface, not a claim that it already exists. The coding agent should implement it and ensure every menu option works.

Example: Dump the current project

```
Select an option [1-6]: 1

[INFO] Project: my-application
[INFO] Scanning directory...
[INFO] Applying exclusion rules...
[INFO] Collecting eligible files...

Files discovered:       48
Files included:         35
Files excluded:         13
Sensitive files skipped: 2

Output: my-application.md

[OK] Markdown export completed successfully.
```

Counts above are illustrative. The actual CLI must display real scan results and never fabricate statistics.

Command reference

The CLI should support both interactive use and direct commands.

1. Dump the current directory

```
codedumper
```

When no arguments are supplied, launch the interactive menu. The user can choose to dump the current directory.

2. Dump a specific project

```
codedumper dump ./my-project
```

Windows example:

```
codedumper dump "D:\Projects\my-project"
```

3. Choose the output file

```
codedumper dump ./my-project --output ./exports/codebase.md
```

4. Preview without exporting

```
codedumper dump ./my-project --dry-run
```

Show the discovered files, exclusions, and estimated export details without writing the Markdown dump.

5. Include hidden files

```
codedumper dump ./my-project --include-hidden
```

Sensitive-file protection must still apply. Hidden files should not automatically bypass security exclusions.

6. Exclude additional directories

```
codedumper dump ./my-project --exclude node_modules --exclude .venv --exclude
dist
```

7. Set a maximum file size

```
codedumper dump ./my-project --max-file-size 1MB
```

The implementation should document supported size units and validate invalid values.

8. Display help

```
codedumper --help
```

9. Display the installed version

```
codedumper --version
```

The commands and flags above define the intended CLI contract. Implement and test them before documenting them as supported features.

What does the generated Markdown look like?

Suppose the project has this structure:

```
my-project/
├── README.md
├── pyproject.toml
├── src/
│   └── app.py
├── tests/
│   └── test_app.py
└── config/
    └── settings.yaml
```

CodeDumper generates `my-project.md`:

```
# Codebase Dump: my-project

## Project Structure

    my-project/
    ├── README.md
    ├── pyproject.toml
    ├── src/
    │   └── app.py
    ├── tests/
    │   └── test_app.py
```

```
    └── config/
        └── settings.yaml

## File: README.md

\`\`\`markdown
# My Project

Project documentation goes here.
\`\`\`

## File: pyproject.toml

\`\`\`toml
[project]
name = "my-project"
\`\`\`

## File: src/app.py

\`\`\`python
def main():
    print("Hello, world!")
\`\`\`

## File: tests/test_app.py

\`\`\`python
def test_example():
    assert True
\`\`\`

## File: config/settings.yaml

\`\`\`yaml
debug: false
\`\`\`
```

The example is illustrative. The real output must contain the actual eligible file contents, not sample content.

CodeDumper must correctly escape Markdown fences when source files contain their own triple-backtick sequences. It should also preserve file paths, text encoding, and file boundaries so the export remains readable and unambiguous.

Safety and exclusion policy

A codebase dump can contain confidential information. CodeDumper should use safe defaults.

By default, the tool should:

- Exclude common virtual environments, caches, dependency folders, build outputs, and Git internals.
- Skip common credential files such as `.env`, private keys, and files matching documented secret-file patterns.
- Avoid collecting common secret-bearing configuration files unless explicitly and safely configured.
- Detect binary files and skip them.
- Enforce configurable per-file size limits.
- Avoid following symbolic links by default, preventing unexpected traversal outside the selected project.
- Never upload project contents or transmit them to an external service.
- Report exclusions clearly without printing secret values.

Important: Filename exclusions alone cannot guarantee that secrets are removed. Users should review generated dumps before sharing them. CodeDumper should never silently promise that an exported codebase is free of credentials.

Technical design

CodeDumper should be implemented as a modular Python application.

A suggested structure:

```
CodeDumper/
├── codedumper/
│   ├── __init__.py
│   ├── cli.py
│   ├── scanner.py
│   ├── tree.py
│   ├── exporter.py
│   ├── exclusions.py
│   ├── safety.py
│   └── config.py
├── tests/
│   ├── test_cli.py
│   ├── test_scanner.py
│   ├── test_tree.py
│   ├── test_exporter.py
│   └── test_safety.py
├── docs/
│   ├── PRD.md
│   ├── ERD.md
│   ├── TRD.md
│   ├── DRD.md
│   └── EDR.md
├── pyproject.toml
├── README.md
├── LICENSE
└── .gitignore
```

The final structure may change after requirements analysis. Do not create unnecessary modules merely to match this example.

Core responsibilities

Module	Responsibility
`cli.py`	Commands, arguments, interactive menu, and terminal output
`scanner.py`	Discover eligible files and directories
`tree.py`	Build a readable project tree
`exporter.py`	Generate the Markdown document
`exclusions.py`	Apply default and user-defined exclusion rules
`safety.py`	Check file types, size limits, links, and sensitive files
`config.py`	Validate settings and configuration

A local file-system CLI does not inherently need a database. Any persistent configuration or other storage must be justified by the requirements.

Development and testing

Install the project for development:

```
python -m pip install -e .
```

Run the test suite:

```
python -m pytest
```

Test the actual CLI:

```
codedumper --help
codedumper --version
codedumper dump . --dry-run
codedumper dump . --output ./test-output.md
```

The project should include tests for:

- Empty directories and nested projects.
- Windows and Unix-style paths.
- Unicode and unusual filenames.
- Binary and oversized files.

- Secret-file exclusions.
- Custom exclusion patterns.
- Permission errors and unreadable files.
- Symbolic links and path traversal.
- Markdown fences inside source files.
- Output creation, overwrite behavior, and invalid arguments.
- Interactive and non-interactive command execution.

Tests must run against temporary test projects and must not expose real credentials or accidentally dump the repository's own secrets.

Roadmap

Version 1.0 — Reliable core CLI

- Directory scanning and tree generation.
- Markdown export.
- Interactive terminal menu.
- Non-interactive commands.
- Exclusion and safety rules.
- Automated tests and installation documentation.

Future releases

- `.codedumperignore` support.
- Multiple export formats.
- Export statistics and token-count estimates.
- Incremental export and content comparison.
- Optional configuration profiles.
- Standalone executables for supported operating systems.

Future features should be implemented only after the core version is reliable.

Contributing

Contributions are welcome.

1. Open an issue describing the problem or proposed improvement.
2. Keep changes focused and modular.
3. Add automated tests for new behavior.
4. Run the test suite before submitting a pull request.
5. Update documentation when commands or behavior change.

Never include credentials, private keys, or confidential project dumps in issues or pull requests.

License

Choose and add an appropriate open-source license before publishing. Do not claim that CodeDumper is released under a license until the repository contains the corresponding license text.

Project philosophy

CodeDumper should make sharing, reviewing, and understanding codebases easier without turning a simple task into a complicated workflow.

One command. One organized document. Your codebase, clearly presented.

```
codedumper
```

Explore your project. Export your code. Understand the whole codebase.